Corrigé : TP CSS3 : Transform, Filtres, Transitions et Animations – 3ème SI

Classe : 3ème SI

Sommaire

Exercice 1 : Transition : le bouton qui réagit au survol
Exercice 2 : Fonctions de progression : linear, ease, ease-in, ease-out
Exercice 3 : Animation avec @keyframes et forwards
Exercice 4 : Plusieurs transform combinés (2 à 4)
Exercice 5 : Filtres CSS : blur, grayscale, invert
Exercice 6 : Plusieurs animations en même temps
Exercice 7 : Formes en mouvement : cercle, carré, triangle
Exercice 8 : La roue qui avance et reste en rotation
Exercice 9 : Projet final : page d'accueil animée (flex, sticky, fixed)

Exercice 1 : Transition : le bouton qui réagit au survol

```
.btn {
  background: #2c3e50;
  color: #fff;
  padding: 12px 24px;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  transition: all 0.4s ease-in-out 0.2s;
}
.btn:hover {
  background: #e74c3c;
  transform: scale(1.1);
}
```

Question 1 : la transition se place dans l'état de base (.btn) pour fonctionner aussi au retour de la souris. Placée dans :hover, l'aller serait progressif mais le retour brusque.

Question 2 : all anime toutes les propriétés modifiées (ici background et transform). background n'anime que la couleur de fond : l'agrandissement (scale) se ferait alors brusquement. Le délai (0.2s) retarde le début de l'effet.

Exercice 2 : Fonctions de progression : linear, ease, ease-in, ease-out

```
@keyframes course {
  from { transform: translateX(0); }
  to   { transform: translateX(500px); }
}
.balle { animation: course 4s linear forwards; }

.linear   { animation-timing-function: linear; }
.ease     { animation-timing-function: ease; }
.ease-in  { animation-timing-function: ease-in; }
.ease-out { animation-timing-function: ease-out; }
```

Positions approximatives (sur 500px) au quart, à la moitié et aux trois quarts de la durée :

Fonction	à 1 s	à 2 s	à 3 s
`linear`	125 px	250 px	375 px
`ease`	204 px	401 px	480 px
`ease-out`	189 px	342 px	453 px
`ease-in`	47 px	158 px	311 px
`ease-in-out`	65 px	250 px	435 px

Question 1 : après 1 s, ease est en tête (≈ 204 px), puis ease-out (≈ 189 px), puis linear (125 px). ease-in est la plus en retard (≈ 47 px).

Question 2 : oui, toutes arrivent à 4 s : la durée est la même. La fonction de progression change seulement la façon de parcourir la distance (la vitesse à chaque instant).

Question 3 : (a) ease-out : le message arrive vite puis se pose en douceur. (b) ease-in : il part doucement puis accélère. (c) linear : vitesse constante.

Bonus : ease-in-out est symétrique (lent, rapide, lent). steps(5) découpe le trajet en 5 paliers brusques.

Exercice 3 : Animation avec @keyframes et forwards

```
@keyframes deplacer {
  from { transform: translateX(0);     background: #3498db; }
  to   { transform: translateX(200px); background: #e74c3c; }
}
.boite { animation: deplacer 2s ease forwards; }

@keyframes apparition {
  from { opacity: 0; transform: translateY(-50px); }
  to   { opacity: 1; transform: translateY(0); }
}
.message {
  opacity: 0;
  animation: apparition 2s ease forwards;
}

.barre { width: 300px; height: 20px; background: #ccc; }
.remplissage {
  width: 0; height: 100%;
  background: #2ecc71;
  animation: charger 3s linear forwards;
}
@keyframes charger {
  from { width: 0%; }
  to   { width: 80%; }
}
```

Question 1 : sans forwards, la boîte revient brusquement à sa position et à sa couleur de départ. Avec forwards, elle reste à droite, en rouge.

Question 2 : forwards conserve le style de la dernière image clé (to / 100 %) après la fin de l'animation.

Question 3 : l'état de départ du message est opacity: 0. Sans forwards, l'élément retrouve ce style à la fin et redevient invisible.

Bonus : avec from { width: 20% } et animation-delay: 1s : sans backwards, la barre reste à 0 pendant le délai ; avec backwards (ou both), elle affiche 20 % dès le début. both = backwards + forwards : 20 % pendant le délai, 80 % à la fin.

Exercice 4 : Plusieurs transform combinés (2 à 4)

```
/* étapes 1 et 2 */
.carte { transition: transform 0.4s; }
.carte:hover {
  transform: translateY(-15px) rotate(5deg) scale(1.1);
}

/* étape 3 */
.carre { transition: transform 1s; }
.repere:hover .ca { transform: translateX(150px) rotate(45deg); }
.repere:hover .cb { transform: rotate(45deg) translateX(150px); }

/* étape 4 */
@keyframes danse {
  0%   { transform: translateX(0)     rotate(0deg)   scale(1)   skewX(0deg); }
  33%  { transform: translateX(150px) rotate(120deg) scale(1.3) skewX(15deg); }
  66%  { transform: translateX(300px) rotate(240deg) scale(0.8) skewX(-15deg); }
  100% { transform: translateX(450px) rotate(360deg) scale(1)   skewX(0deg); }
}
.danseur { animation: danse 5s ease-in-out forwards; }
```

Question 1 : les fonctions s'appliquent l'une après l'autre, dans l'ordre d'écriture, chacune dans le repère déjà transformé par les précédentes. A glisse de 150px vers la droite puis pivote sur place. B pivote d'abord de 45° : son axe horizontal est alors incliné, donc translateX(150px) l'envoie en diagonale vers le bas et la droite.

Question 2 : le navigateur calcule les valeurs intermédiaires fonction par fonction. Il faut donc les mêmes fonctions, dans le même ordre, à chaque étape ; sinon l'animation devient brusque ou incorrecte.

Bonus : avec transform-origin: bottom left, la rotation et le scale se font autour du coin inférieur gauche au lieu du centre.

Exercice 5 : Filtres CSS : blur, grayscale, invert

```
.photo { transition: filter 0.5s; }

.p1 { filter: blur(4px); }
.p1:hover { filter: blur(0); }

.p2 { filter: grayscale(100%); }
.p2:hover { filter: grayscale(0%); }

.p3:hover { filter: invert(100%); }

.p4 { filter: grayscale(100%) blur(2px); }
.p4:hover { filter: none; }

@keyframes flou-net {
  from { opacity: 0; filter: blur(12px); }
  to   { opacity: 1; filter: blur(0); }
}
.galerie { animation: flou-net 2s ease forwards; }
```

Question 1 : plus la valeur en px est grande, plus l'élément est flou ; blur(0) donne un élément net.

Question 2 : grayscale(50%) atténue les couleurs (mi-gris) ; grayscale(100%) donne du noir et blanc complet.

Question 3 : invert(100%) donne du noir pour le blanc, du blanc pour le noir, et la couleur complémentaire pour le rouge (un cyan).

Question 4 : le filtre s'applique à l'élément entier : fond, texte et enfants.

> **Bonus :** invert(50%) donne un gris uniforme, quelle que soit la couleur d'origine (la moitié entre la couleur et son inverse).

Exercice 6 : Plusieurs animations en même temps

```
@keyframes traverser {
  from { transform: translateX(0); }
  to   { transform: translateX(500px); }
}
@keyframes rougir {
  from { background-color: #f1c40f; }
  to   { background-color: #e74c3c; }
}
@keyframes lueur {
  from { box-shadow: 0 0 10px #f1c40f; }
  to   { box-shadow: 0 0 40px #f39c12; }
}
@keyframes grossir {
  from { transform: scale(1); }
  to   { transform: scale(1.5); }
}

/* étapes 2 à 3 : sur .soleil */
.soleil {
  animation: traverser 6s linear forwards,
             rougir 6s linear forwards,
             lueur 1s ease-in-out infinite alternate;
}

/* étape 5 : correction avec deux éléments */
.orbite {
  position: absolute;
  bottom: 20px; left: 10px;
  width: 70px; height: 70px;
  animation: traverser 6s linear forwards;
}
.soleil {
  width: 100%; height: 100%;
  border-radius: 50%;
  background: #f1c40f;
  animation: grossir 6s linear forwards,
             rougir 6s linear forwards,
             lueur 1s ease-in-out infinite alternate;
}
```

HTML de la correction :

Observation : le soleil traverse le ciel, change de couleur (jaune vers rouge) et sa lueur pulse en continu. À la fin, il reste à droite et rouge (forwards), la lueur continue (infinite).

Question 1 : traverser et grossir modifient la même propriété (transform). Quand deux animations agissent sur la même propriété, celle qui est la dernière dans la liste l'emporte : seul le scale est appliqué et le déplacement est écrasé.

Bonus : lueur 1s ease-in-out 1s infinite alternate : la lueur commence après 1 s ; chaque animation de la liste a sa propre durée, son propre délai et sa propre répétition.

Exercice 7 : Formes en mouvement : cercle, carré, triangle

```
@keyframes droite {
  from { transform: translateX(0); }
  to   { transform: translateX(520px); }
}
@keyframes aller-retour {
  from { transform: translateX(0) rotate(0deg); }
  to   { transform: translateX(520px) rotate(180deg); }
}
@keyframes zigzag {
  0%   { transform: translate(0, 0); }
  33%  { transform: translate(170px, -60px); }
  66%  { transform: translate(350px, 0); }
  100% { transform: translate(520px, -60px); }
}

.cercle   { animation: droite 3s linear forwards; }
.carre2   { animation: aller-retour 2s ease-in-out infinite alternate; }
.triangle { animation: zigzag 4s linear forwards; }
```

Bonus : animation-delay: 0s sur .cercle, 1s sur .carre2 et 2s sur .triangle (avec both si on veut que la forme prenne le style de départ pendant l'attente).

Exercice 8 : La roue qui avance et reste en rotation

```
@keyframes avancer {
  from { transform: translateX(0); }
  to   { transform: translateX(480px); }
}
@keyframes tourner {
  from { transform: rotate(0deg); }
  to   { transform: rotate(360deg); }
}

.support { animation: avancer 4s linear forwards; }
.roue    { animation: tourner 1s linear infinite; }
```

Observation : le support s'arrête à 480px (forwards), mais la roue continue de tourner (infinite).

Question 1 : les deux animations utilisent la propriété transform. Sur le même élément, la seconde écrase la première : la roue ne ferait que tourner ou que avancer. Avec deux éléments imbriqués, chaque animation a son propre transform.

Bonus : nombre de tours = 480 ÷ 314 ≈ 1,53 tour en 4s, donc durée d'un tour ≈ 4 ÷ 1,53 ≈ 2,6s (animation: tourner 2.6s linear infinite).

Exercice 9 : Projet final : page d'accueil animée (flex, sticky, fixed)

```
.entete {
  background: #2c3e50;
  padding: 15px 30px;
  display: flex;
  justify-content: space-between;
  align-items: center;
  position: sticky;
  top: 0;
  z-index: 10;
}
.menu { display: flex; gap: 20px; }
.menu a {
  color: #fff;
  text-decoration: none;
  transition: color 0.3s, transform 0.3s;
}
.menu a:hover {
  color: #f1c40f;
  transform: translateY(-3px);
}

.cartes {
  display: flex;
  gap: 20px;
  flex-wrap: wrap;
  justify-content: center;
}
.carte2 { transition: transform 0.3s, background 0.3s; }
.carte2:hover {
  transform: scale(1.08);
  background: #2c3e50;
}

@keyframes apparition {
  from { opacity: 0; transform: translateY(-40px); }
  to   { opacity: 1; transform: translateY(0); }
}
.titre {
  opacity: 0;
  animation: apparition 1.5s ease forwards;
}

@keyframes montrer {
  from { opacity: 0; }
  to   { opacity: 1; }
}
.haut {
  position: fixed;
  bottom: 20px;
  right: 20px;
  opacity: 0;
  animation: montrer 1s ease 2s forwards;
}
```

Question 1 : flex-wrap: wrap autorise les éléments à passer à la ligne quand la place manque. Sans lui, les trois cartes restent sur une seule ligne et se serrent ou débordent.

Question 2 : sans top: 0 (ou une autre valeur), sticky n'a pas de seuil : l'en-tête ne colle jamais et défile comme un élément normal.

Question 3 : transform ne s'applique pas aux éléments en ligne (inline). Les enfants directs d'un conteneur flex sont « blockifiés » : les liens se comportent comme des blocs, donc transform fonctionne.

Question 4 : sticky reste dans le flux de la page et ne colle qu'à partir du seuil (top: 0), dans son parent. fixed est sorti du flux et reste toujours au même endroit de la fenêtre. Sans forwards, le bouton redeviendrait invisible (opacity: 0) à la fin de l'animation, car l'état de départ est opacity: 0.

Étape 5 (z-index) : le titre animé garde un transform avec forwards, ce qui le place au-dessus de l'en-tête au défilement ; z-index: 10 remet l'en-tête au premier plan.

> **Bonus :** html { scroll-behavior: smooth; } et @keyframes rebondir { from { transform: translateY(0); } to { transform: translateY(-10px); } } avec animation: montrer 1s ease 2s forwards, rebondir 1s ease-in-out 3s infinite alternate; : les deux animations peuvent être sur le même élément car elles n'utilisent pas la même propriété (opacity et transform).

Enseignant : Aymen Essouyah